

Kapselung (Encapsulation)

Aspekt	Beschreibung	Java-spezifische Konzepte/Techniken
Definition	Kapselung ist ein Grundprinzip der objektorientierten Programmierung (OOP), das den direkten Zugriff auf interne Daten einer Klasse einschränkt.	Nur über definierte Methoden (Getter/Setter oder Business-Methoden) kann auf die Daten zugegriffen werden. Verbessert die Sicherheit und Wartbarkeit des Codes.
Schutz interner Zustände	Durch Kapselung wird verhindert, dass Variablen oder Zustände direkt von außen verändert oder ausgelesen werden können.	Zugriffskontrolle erfolgt durch private Felder und öffentliche Methoden zur Datenmanipulation. Schutz vor unerwarteten Zustandsänderungen und Inkonsistenzen.
Zugriffsmodifizierer	Java bietet verschiedene Modifizierer (<code>private</code> , <code>protected</code> , <code>public</code> , <code>package-private</code>), um den Zugriff auf Attribute und Methoden zu kontrollieren.	<code>private</code> : Zugriff nur innerhalb der Klasse. <code>protected</code> : Zugriff innerhalb des Pakets und durch Unterklassen. <code>public</code> : Vollständiger Zugriff. Kein Modifizierer: Paketzugriff.
Datenhiding (Datenverbergung)	Das Verbergen von Implementierungsdetails verhindert ungewollte Abhängigkeiten und erleichtert Änderungen an der internen Implementierung.	Innere Datenstrukturen können geändert werden, ohne dass andere Klassen betroffen sind. Erhöht die Robustheit, indem der externe Zugriff begrenzt wird.
Getter und Setter	Bereitstellung kontrollierter Zugriffsmethoden anstelle direkter Feldmanipulation zur Kontrolle von Lese- und Schreibzugriffen.	<code>getX()</code> für das Lesen von Attributen. <code>setX(value)</code> für das Schreiben mit optionaler Validierung. Ermöglicht Berechnungen oder Transformationen beim Zugriff.
Unveränderlichkeit (Immutability)	Durch Kapselung kann ein Objekt so gestaltet werden, dass es nach der Initialisierung nicht mehr verändert werden kann.	Vermeidung von Settern für unveränderliche Felder. Initialisierung durch Konstruktoren. Verwendung von <code>final</code> für Variablen.
Final-Modifikator für Sicherheit	<code>final</code> kann verwendet werden, um sicherzustellen, dass Referenzen oder Methoden nicht unerwartet verändert oder überschrieben werden.	<code>final</code> für Felder bedeutet, dass sie nach der Initialisierung nicht geändert werden können. <code>final</code> Methoden können nicht von Unterklassen überschrieben werden.

Aspekt	Beschreibung	Java-spezifische Konzepte/Techniken
Encapsulation und Konstruktoren	Konstruktoren dienen zur Initialisierung von Objekten und können mit spezifischen Einschränkungen versehen werden, um Konsistenz zu gewährleisten.	Pflichtparameter können über Konstruktoren gesetzt werden. Verhindert nicht zulässige Objektzustände. Konstruktor-Überladung für verschiedene Initialisierungsszenarien.
Datenvalidierung	Kapselung ermöglicht Validierungen in den Settermethoden, um ungültige Zustände zu verhindern.	Beispiel: Ein <code>setAge(int age)</code> könnte sicherstellen, dass <code>age > 0</code> ist. Verhindert fehlerhafte Eingaben durch externe Klassen.
Thread-Sicherheit durch Kapselung	Durch das Verbergen von Zuständen und die Bereitstellung synchronisierter Zugriffsmethoden kann Thread-Sicherheit verbessert werden.	Synchronisierte Methoden (<code>synchronized</code>) oder <code>volatile</code> für sichere nebenläufige Zugriffe. Reduktion von Race Conditions durch private Felder und kontrollierten Zugriff.
Encapsulation und Datenkapselung mit Records	Java 14 führte <code>record</code> -Klassen ein, die standardmäßig Felder als <code>private final</code> kapseln und automatische Getter bereitstellen.	<code>record Person(String name, int age) {}</code> generiert automatisch <code>private final</code> Felder mit öffentlichen Getter-Methoden. Reduziert Boilerplate-Code und fördert Unveränderlichkeit.
Verhinderung direkter Objektmanipulation	Eine Klasse kann Methoden bereitstellen, die intern bestimmte Abläufe durchführen, anstatt externe Manipulation zuzulassen.	Direkte Feldänderungen werden vermieden. Beispiel: <code>account.deposit(amount)</code> statt <code>account.balance += amount</code> .
Kapselung in APIs	Public APIs kapseln interne Implementierungsdetails, um Änderungen zu erleichtern und Kompatibilität zu bewahren.	Interne Implementierungsdetails sind versteckt. Die API bleibt stabil, auch wenn sich die interne Logik ändert.
Kapselung und Vererbung	Kapselung beeinflusst die Sichtbarkeit von geerbten Eigenschaften und Methoden.	<code>private</code> Mitglieder der Superklasse sind in der Unterklasse nicht sichtbar. <code>protected</code> erlaubt Vererbung, aber nicht öffentlichen Zugriff.
Encapsulation und JavaBeans-Konvention	JavaBeans sind Klassen, die private Felder und öffentliche Getter/Setter bereitstellen, um Kapselung zu gewährleisten.	Beispiel: <code>private String name;</code> <code>public String getName() { return name; }</code> <code>public void setName(String name) { this.name = name; }</code>

Aspekt	Beschreibung	Java-spezifische Konzepte/Techniken
Reduzierung von Abhängigkeiten	Durch das Verstecken interner Implementierungsdetails wird die Kopplung zwischen Klassen reduziert.	Änderungen an einer Klasse haben weniger Auswirkungen auf andere Klassen. Förderung des Low Coupling & High Cohesion-Prinzips.
Encapsulation und das SOLID-Prinzip	Kapselung unterstützt das Single Responsibility Principle (SRP) und das Open/Closed Principle (OCP) aus den SOLID-Prinzipien.	SRP: Jedes Objekt sollte nur für seine eigenen Daten verantwortlich sein. OCP: Interne Implementierungen können geändert werden, ohne andere Klassen zu beeinflussen.
Encapsulation und Sicherheitsaspekte	Sensible Daten können vor unberechtigtem Zugriff geschützt werden, indem sie gekapselt und nur über kontrollierte Mechanismen zugänglich gemacht werden.	Beispiel: Passwort-Hashing in einer Methode statt direkter Speicherung von Passwörtern. Begrenzung der Sichtbarkeit von sicherheitskritischen Methoden.
Encapsulation und Refactoring	Kapselung erleichtert das Refactoring, da Änderungen an internen Implementierungsdetails keine Auswirkungen auf externe Klassen haben.	Methoden können ohne Änderung externer Klassen umstrukturiert werden. Verbesserungen wie Caching oder Logging können hinzugefügt werden, ohne die öffentliche API zu verändern.

Kapselung in Java ist ein fundamentales Prinzip, das sicherstellt, dass die interne Struktur eines Objekts vor äußeren Zugriffen geschützt ist. Nur vordefinierte Schnittstellen (Methoden) sind zugänglich.

- **Zugriffsmodifikatoren** in Java steuern den Zugriff auf Felder und Methoden:
 - **private**: Nur innerhalb der Klasse zugänglich.
 - **protected**: Innerhalb der Klasse, des Pakets und in Unterklassen zugänglich.
 - **public**: Überall zugänglich.
 - **default (Package-Private)**: Nur innerhalb des gleichen Pakets zugänglich.
- **Getter- und Setter-Methoden**: Dies sind Methoden, die den Zugriff auf private Felder ermöglichen. Sie kontrollieren den Zugriff auf Daten und verhindern, dass diese direkt verändert werden, was zu unvorhersehbarem Verhalten führen könnte. Getter sind oft einfache Methoden, die den Wert eines privaten Feldes zurückgeben, während Setter die Möglichkeit bieten, den Wert zu setzen und Validierungslogik anzuwenden.
- **Datenvalidierung** innerhalb von Setter-Methoden stellt sicher, dass nur gültige Daten einem Objekt zugewiesen werden. Dies erhöht die Datenintegrität und schützt vor fehlerhaften Zuständen.

Zusammenfassung

Kapselung ist ein essenzielles Konzept in Java, das den Zugriff auf interne Daten schützt, die Sicherheit und Wartbarkeit von Code verbessert und eine klare Trennung zwischen Implementierung und Schnittstelle ermöglicht. Sie hilft, fehlerhafte Zustände zu vermeiden, ermöglicht eine kontrollierte Datenmanipulation und reduziert Abhängigkeiten zwischen Klassen.

Prof. Dr. Carsten Müller | Software Engineering